Computer Graphics

Visibility

Dr. Hardik Jain
Assistant Professor
Department of Computer Science and Engineering
IIT Jodhpur

1

Visible Surface Determination
Hidden Surface Elimination

Given a set of 3D objects and a viewing specification, determine which lines or surfaces of the objects are visible at each pixel on the projection plane

2

Visible Surface Determination (VSD)

- Multiple object points can project to the same image pixel.
  - Which one gives the colour to the pixel?
- Although the idea is simple, it requires significant processing and compute.
- Need to analyse the scene from the camera.
  - Centre of projection for Perspective Projection
  - Direction of projection for Parallel Projection
- Two classic categories of algorithms:
  - Object-space
  - Image-space

Object Precision Algorithm


```
for (each object in the world) {
   Determine unobstructed parts of the object
   Draw those parts with appropriate colours
}
```

- Compares each of the n objects to itself and other objects, with a computational effort of n^2
- Performs at the precision with which each object is defined and the determines the visibility at each object.
- Works in the original continuous object space.
- Since the calculations are done without regard to a particular display resolution, it is followed by drawing step. Only this step is redone if the image size changes.

Image Precision Algorithm


```
for (each pixel in the image) {
   Determine closest object in the direction of projector
   Draw the pixel with appropriate colours
}
```

- For each of the p pixels all the n objects need to be examined, with effort proportional to np.
- Performs at the resolution of display device and determines the visibility at each pixel.
- Works in the discrete image space.
- A natural choice for raster graphics.
- Image resizing involves repeating the entire work.

5

Visible Surface Determination (VSD)

Object Precision Algorithm	Image Precision Algorithm
• Operates on continuous object data	• Operates on sampled data
• No problem of aliasing	• Problem of aliasing in computing visibility
• Developed for vector graphics	• Developed for raster devices
• Best suited for accuracy	• Best suited for speed

Modern algorithms combine both

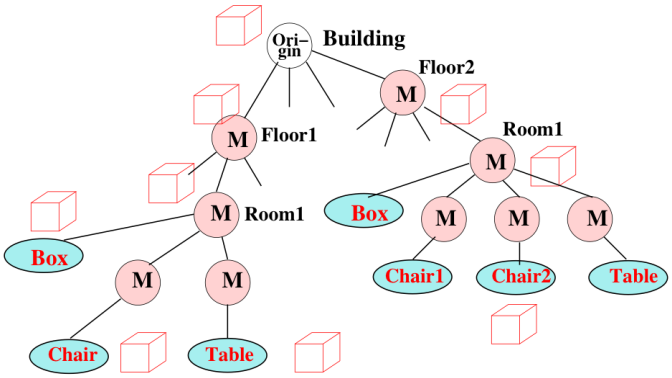
6

VSD Algorithm Classification

- **Broad Object Level**:
  - View Frustrum Culling
- **Medium Primitive Level**:
  - Back-face culling
- **Fine Pixel Level**:
  - Z-buffer (Depth Buffer)
  - Depth Sort (Painter's Algorithm)
  - Binary Spanning Tree Algorithm
  - Ray Casting

Scene Hierarchy

- **Leaf Level**: Object bounding box, for eg: table, chair
- **Intermediate Level**: Bounding box that encloses the child nodes and covers geometry below it.
- **Root Level**: BB covers the whole scene.



View Frustum Culling

Object Level VSD

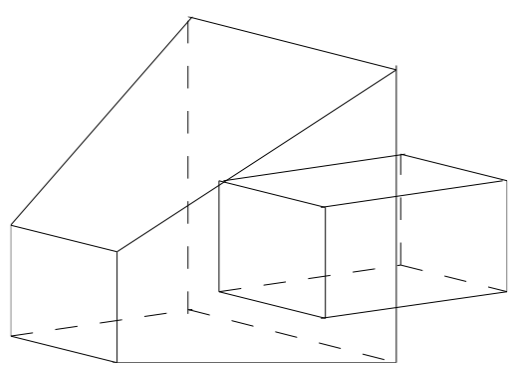
- View Frustum Culling Eliminate objects that are outside the view volume.
- Large parts of the scene will be eliminated this way.

```
Compare bounding volume of node with view volume:
   if (intersection is null): eliminate tree from root
   if (volume contained in frustum): render without clipping
   else: recursively check for each child of node till leaf
```

- Object hierarchy really helps.
  - Whole campus, each building, each floor, each room, etc., in the hierarchy.

Box-Frustum Intersection

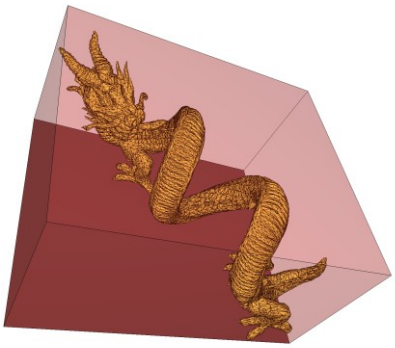
- **Orthographic**: Intersection of two boxes. Simultaneous X-Y-Z overlaps.
- **Perspective**: Clip against 6 planes of the frustum.



Types of object bounding boxes

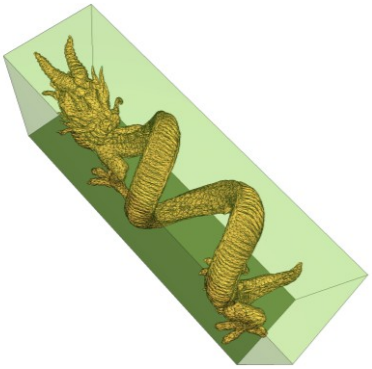
Axis Aligned Bounding Box (AABB)

Easy to find the box, but not efficient



Oriented Bounding Box (OBB)

More accurate, but more computation



Types of object bounding boxes

Axis Aligned Bounding Box (AABB)

- In ORC, find separately the minimum and maximum values of X, Y, and z

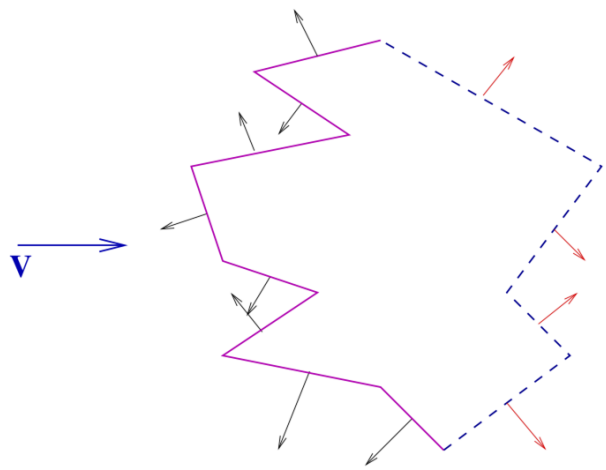
Oriented Bounding Box (OBB)

- Find the smallest bounding volume: somewhat involved
- Find the major/minor axes for the shape and fit a box

Back Face Culling

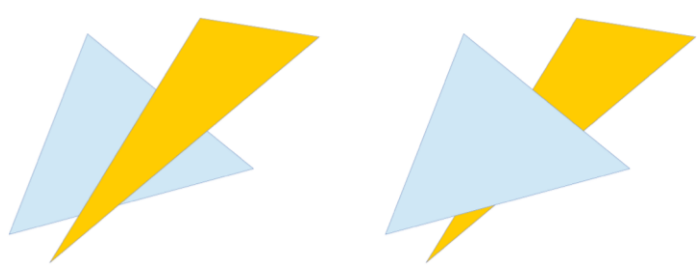
Medium Level VSD

- Only the front side can be seen of solid, opaque objects.
- If surface normals point out of the object, polygons with normals pointing away from the viewer cannot be seen.
- Let v be the vector from COP to any point on the polygon and n be the surface normal.
- If $v \cdot n < 0$, draw. Else discard.



Which of the Two Cases ?

- Parts of one primitive is in front.
- Need correct picture Back-to-front drawing gives correct picture, when possible
- When part of the object is infront and part of it is behind: Fine pixel level VSD has to be performed



Z-buffer or Depth-buffer Algorithm

Pixel Level VSD

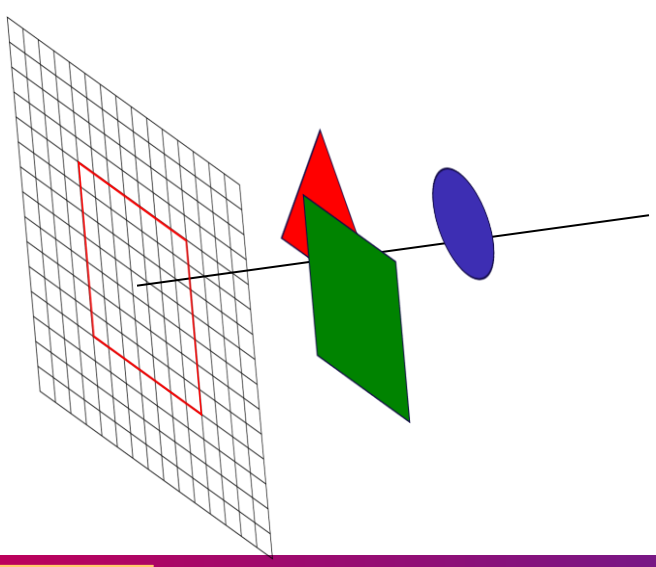
Pseudo Code

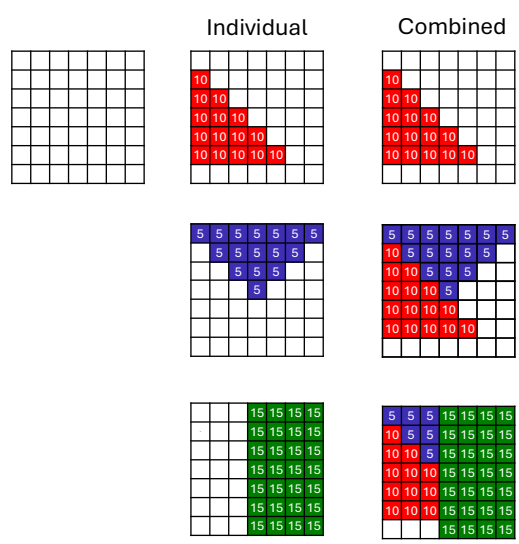
```
init depthBuff (x, y) = 1.0;
init frameBuff (x, y) = backgndColor;
for (each polygon P) {
    for (each pixel (x,y) in the P's Projection) {
        z = interpolated_z_at(P, x, y)
        if (z > depthBuff[x,y]) {
            depthBuff[x,y] = z
            frameBuff[x,y] = shade(P)
        }
    }
}
```

- An image-precision algorithm that needs a z or depth-buffer parallel to the frame buffer.
- Larger *z* implies a closer 3D point in any direction
- Write to frame buffer and *z*-buffer only when the *z* value is larger than previously stored value.

15

Z-buffer or Depth-buffer Algorithm





16

Depth-buffer Manipulation

- Each polygon is scan converted into buffers. With each pixel representing the z value and the corresponding polygon colour.
- Then the buffers could be combined based on z value and the color is obtained from highest z value

10	10	8	6	4
10	10	8	6	
10	8	6	4	
10	8	6	4	
10	8	6	2	
10	8	2	1	

	10	8	6	2
			4	2
			6	2
			6	2
	8	6	6	4
	8	6	4	2

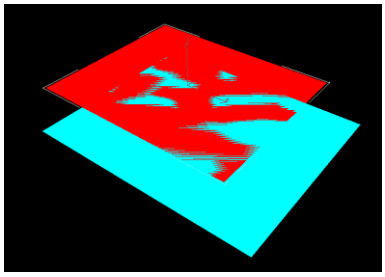
		4	8		
4	4	4	6	8	8
4	4	4	8	8	8
15	12	10	8	6	4

10	10	10	8	6	2
10	10	8	6	4	2
10	8	6	8	6	2
10	8	6	4	8	8
10	8	8	8	8	8
15	12	10	8	6	4

Coplanar Surfaces

Z-fighting

- A rendering artifact where two coplanar or nearly coplanar surfaces flicker or jitter as they compete for pixels in the z-buffer, due to floating-point precision limits.



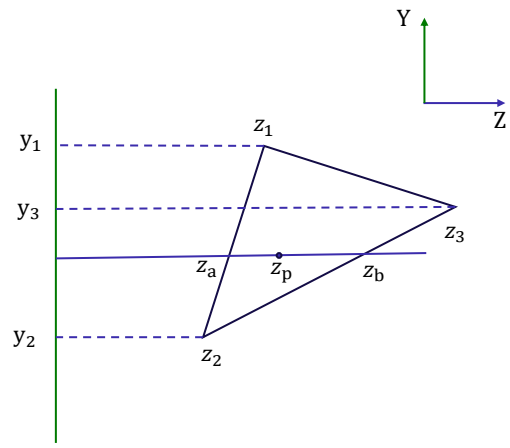
z-shimmering

- Dynamic variant, often seen as noisy flickering when the camera moves, because tiny depth differences become indistinguishable after perspective projection

Possible Fixes: Shrink near/far ratio, Depth bias/offset, Higher Precision

Computing Depth for Interior Points

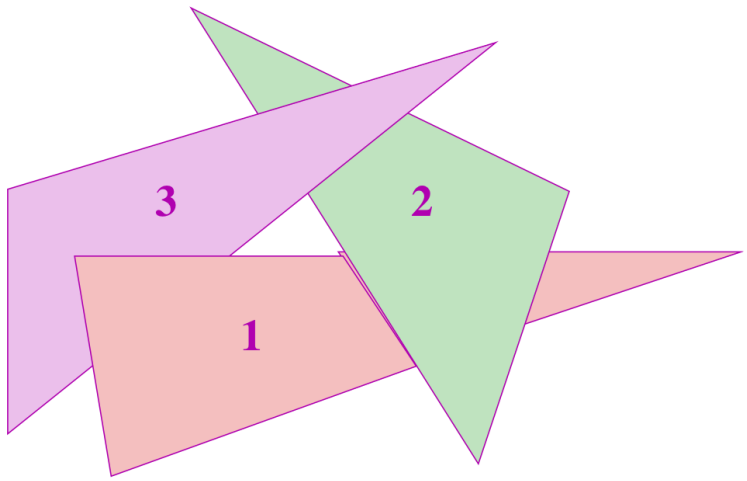
- Exploit coherence in depth values over the plane to compute interior z values given the vertex values.
- $z_a = z_1 + (z_2 - z_1)\frac{(y_a - y_1)}{(y_2 - y_1)}$
- $z_b = z_2 + (z_3 - z_2)\frac{(y_b - y_2)}{(y_3 - y_2)}$
- $z_p = z_a + (z_b - z_a)\frac{(x_p - x_a)}{(x_b - x_a)}$
- When x or y increments by 1 along a side or a scan line, z changes by a constant Δz.
- $\Delta z = \frac{(z_2 - z_1)}{(y_2 - y_1)}$ along edge 12
- $\Delta z = \frac{(z_b - z_a)}{(x_b - x_a)}$ along scan line



Z-buffering: Discussion

- Any shape with per pixel z can be handled correctly.
- Time is independent of number of primitives.
  - As for each primitive only primitive covering pixels are compared with the Z-buffer
  - Fixed for fixed resolution: Adding primitives doesn't multiply work pairwise
- Easy to implement; can do with a single scan-line Z-buffer.
- Z-buffer can be read back and saved.
- Needs extra memory, but memory is cheap.
- Can cause aliasing or z-fighting (shimmering).

A Difficult Case for Ordering



21

Thank You

Anonymous Course Feedback

22